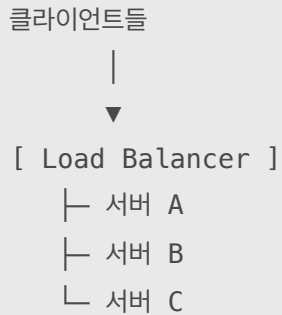


네트워크 8 - 로드밸런싱

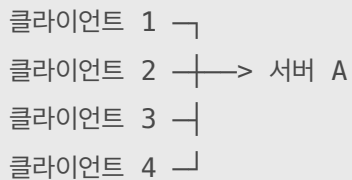
남궁명수

[Load Balancing]

로드 밸런싱(Load Balancing)은 클라이언트의 요청이나 네트워크 연결을 여러 서버에 분산하여 특정 서버에 부하가 집중되지 않도록 하는 기술이다.



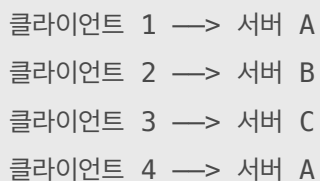
로드 밸런서를 사용하지 않으면 모든 요청이 서버 한 대에 집중된다.



서버 A의 처리 능력을 초과하면 다음 문제가 발생할 수 있다.

- 응답 시간 증가
- 요청 실패
- 서버 과부하
- 서비스 장애
- 단일 서버 장애로 전체 서비스 중단

로드 밸런서를 사용하면 요청을 여러 서버에 나눌 수 있다.

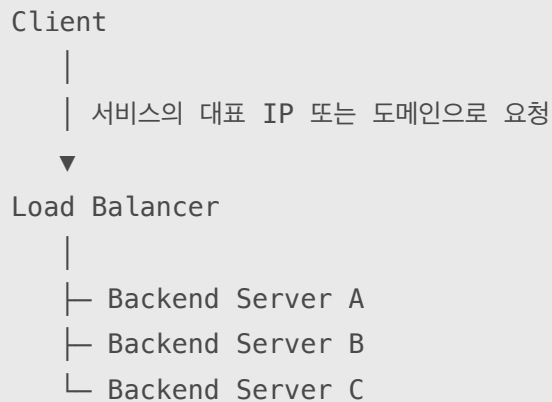


[1] [로드 밸런서의 역할]

로드 밸런서는 단순히 요청을 나누는 것 외에도 다음 기능을 제공할 수 있다.

- 트래픽 분산
- 서버 상태 확인
- 장애 서버 제외
- TLS 종료
- 경로 및 도메인 기반 라우팅
- 세션 유지
- 요청 제한
- 접근 제어
- 로그 수집
- 서버 확장 및 축소 지원

전체 구조는 일반적으로 다음과 같다.



클라이언트는 일반적으로 Backend 서버의 주소를 직접 알 필요가 없다.

클라이언트가 아는 주소: `service.example.com`

Backend 실제 주소:

`10.0.0.11:8080`

`10.0.0.12:8080`

`10.0.0.13:8080`

[2] [로드 밸런서 전체 과정]

사용자가 다음 주소에 접속한다고 하자.

```
https://service.example.com/users
```

[1단계: DNS Lookup]

```
service.example.com
```

→ Load Balancer의 IP 주소 반환

DNS는 일반적으로 Backend 서버의 사설 IP가 아니라 외부 요청을 받을 로드 밸런서 주소를 반환한다.

[2단계: 클라이언트가 로드 밸런서에 연결]

```
클라이언트
```

→ Load Balancer의 443번 포트에 연결

[3단계: 로드 밸런서가 Backend 선택]

로드 밸런서는 설정된 알고리즘을 사용하여 서버를 선택한다.

```
Round Robin
```

```
Least Connection
```

```
Weighted Round Robin
```

```
IP Hash
```

[4단계: Backend로 전달]

```
Load Balancer
```

→ 선택된 Backend 서버로 요청 전달

[5단계: 응답 반환]

```
Backend 서버
```

→ Load Balancer

→ 클라이언트

클라이언트 관점에서는 로드 밸런서와 통신하지만, 실제 요청 처리는 여러 Backend 서버 중 하나가 담당한다.

[3] [Round Robin]

Round Robin은 Backend 서버를 순서대로 번갈아 선택하는 가장 단순한 로드 밸런싱 알고리즘이다.
서버가 세 대라면:

요청 1 → 서버 A
요청 2 → 서버 B
요청 3 → 서버 C
요청 4 → 서버 A
요청 5 → 서버 B
요청 6 → 서버 C

A → B → C → A → B → C ...

[동작 예시]

Backend 목록:

A, B, C

현재 선택 위치:

A

다음 요청:

B

다음 요청:

C

다음 요청:

다시 A

Round Robin은 서버의 현재 연결 수나 처리 중인 작업량을 확인하지 않고 순서만 보고 서버를 선택한다.

[장점]

- 구현이 단순함
- 서버 선택 비용이 작음
- 서버 성능과 요청 처리 시간이 비슷한 환경에서 고르게 분산됨
- 별도의 복잡한 상태 정보를 적게 관리함

[단점]

Round Robin은 요청의 실제 처리 비용을 고려하지 않는다.
예를 들어:

요청 1 → 서버 A → 처리 시간 30초
요청 2 → 서버 B → 처리 시간 1초
요청 3 → 서버 C → 처리 시간 1초
요청 4 → 서버 A → 처리 시간 20초

요청 개수는 고르게 배분했지만 서버 A에는 무거운 요청이 쌓일 수 있다.

또한 서버 성능이 서로 다를 때도 문제가 발생한다.

서버 A: CPU 8코어
서버 B: CPU 4코어
서버 C: CPU 2코어

일반 Round Robin은 세 서버에 같은 비율로 요청을 전달하므로 성능이 낮은 서버가 먼저 과부하될 수 있다.

[4] [Weighted Round Robin]

서버 성능이 서로 다르면 가중치를 적용할 수 있다.

서버 A: Weight 3
서버 B: Weight 2
서버 C: Weight 1

단순화하면 요청을 다음 비율로 전달한다.

A, A, A, B, B, C

즉:

서버 A → 전체 요청의 약 3/6
서버 B → 전체 요청의 약 2/6
서버 C → 전체 요청의 약 1/6

서버의 CPU, 메모리, 성능 또는 처리 능력을 고려하여 가중치를 설정할 수 있다.

[장점]

- 성능이 다른 서버에 적합
- 구현과 이해가 비교적 단순
- 점진적인 서버 교체나 테스트에 활용 가능

[단점]

- 서버의 실시간 부하를 직접 반영하지는 않음
- 적절한 가중치를 운영자가 정해야 함
- 요청별 처리 시간이 크게 다르면 불균형이 생길 수 있음

[5] [Least Connection]

Least Connection은 현재 활성 연결 수가 가장 적은 서버를 선택하는 방식이다.

예를 들어:

서버 A: 활성 연결 10개

서버 B: 활성 연결 3개

서버 C: 활성 연결 7개

새 연결은 서버 B로 전달한다.

새 연결 → 서버 B

연결이 추가되면:

서버 A: 10개

서버 B: 4개

서버 C: 7개

다음 연결에서도 가장 적은 서버가 선택된다.

[장점]

- 서버의 현재 연결 상태를 반영
- 연결 유지 시간이 서로 다른 서비스에 유리
- 장시간 연결이 많은 환경에서 Round Robin보다 균형 잡힌 분산 가능
- WebSocket, 스트리밍, 장기 TCP 연결 등에 활용 가능

[단점]

Least Connection도 서버의 실제 CPU 사용량이나 요청 처리 비용을 정확하게 아는 것은 아니다.

서버 A:

연결 5개, 모두 무거운 작업

서버 B:

연결 10개, 모두 가벼운 대기 작업

연결 수만 보면 서버 A가 더 여유 있어 보이지만 실제 CPU 부하는 서버 A가 더 높을 수 있다.
또한 HTTP Keep-Alive나 HTTP/2에서는 하나의 연결 안에서 여러 요청을 처리할 수 있다.

TCP 연결 1개

└ HTTP/2 Stream 100개

따라서 단순 연결 수가 실제 요청량과 정확히 일치하지 않을 수 있다.

[6] [Weighted Least Connection]

서버 성능과 현재 연결 수를 함께 고려하는 방식이다.

예를 들어 서버마다 가중치를 설정한다.

서버 A: Weight 4, 현재 연결 20

서버 B: Weight 2, 현재 연결 8

서버 C: Weight 1, 현재 연결 6

로드 밸런서는 연결 수뿐 아니라 각 서버가 처리할 수 있는 상대적인 용량까지 고려해 서버를 선택한다.

개념적으로는 다음과 같은 판단을 한다.

현재 연결 부하 / 서버 처리 능력

정확한 계산 방식은 로드 밸런서 구현에 따라 다르다.

[7] [Round Robin과 Least Connection 비교]

구분	Round Robin	Least Connection
선택 기준	서버 순서	현재 활성 연결 수
상태 확인	거의 필요 없음	연결 수 관리 필요
구현 복잡도	단순	상대적으로 복잡
요청 처리 시간이 비슷한 경우	적합	적합
연결 시간이 서로 다른 경우	불균형 가능	상대적으로 유리
서버 실시간 상태 반영	거의 하지 않음	연결 수 기준으로 반영
HTTP Keep-Alive·HTTP/2	요청량과 불일치 가능	연결 수와 실제 작업량 불일치 가능
대표 장점	단순하고 예측 가능	장기 연결 환경에서 균형 개선

핵심 차이는 다음과 같다.

Round Robin → 다음 순서의 서버를 선택

Least Connection → 현재 연결이 가장 적은 서버를 선택

[8] [Health Check]

로드 밸런서는 요청을 분배하기 전에 Backend 서버가 정상인지 확인해야 한다.
이를 Health Check라고 한다.

Load Balancer

|
├ 서버 A: 정상
├ 서버 B: 비정상
└ 서버 C: 정상

서버 B가 비정상이라면 요청 분배 대상에서 제외한다.

요청 1 → 서버 A
요청 2 → 서버 C
요청 3 → 서버 A

서버 B가 다시 정상 상태가 되면 분배 대상에 포함할 수 있다.

[L4 Health Check]

특정 TCP 포트에 연결할 수 있는지 확인한다.

10.0.0.11:8080으로 TCP 연결 가능?

장점은 단순하고 빠르다는 것이다.

하지만 포트가 열려 있다는 것만으로 애플리케이션이 정상이라는 사실까지 보장하지는 않는다.

TCP 연결 가능
하지만 데이터베이스 연결 실패
→ 실제 서비스는 정상 동작하지 않을 수 있음

[L7 Health Check]

HTTP 요청을 보내 응답 상태와 내용을 확인한다.

GET /health HTTP/1.1
Host: backend

응답:

HTTP/1.1 200 OK

{"status":"healthy"}

단순 포트 연결뿐 아니라 애플리케이션이 실제 요청을 처리할 수 있는지 확인할 수 있다.

예를 들어 다음 항목을 포함할 수 있다.

- 애플리케이션 실행 상태
- 데이터베이스 연결 상태
- 필수 외부 서비스 상태
- 디스크 또는 메모리 상태

다만 너무 많은 외부 의존성을 Health Check에 포함하면 일시적인 외부 장애로 모든 Backend가 동시에 제외될 수 있으므로 신중하게 설계해야 한다.

[9] [L4 Load Balancing]

L4 로드 밸런싱은 OSI 4계층인 전송 계층의 정보를 기준으로 연결을 분산한다.
주로 다음 정보를 확인한다.

Source IP
Source Port
Destination IP
Destination Port
Protocol: TCP 또는 UDP

즉, 4-Tuple 또는 프로토콜까지 포함한 5-Tuple 정보를 기준으로 Backend를 선택한다.

TCP
192.168.0.10:53000
↓
203.0.113.10:443

L4 로드 밸런서는 일반적으로 HTTP 메시지의 상세 내용을 기준으로 라우팅하지 않는다.

확인 가능:

IP 주소
포트 번호
TCP 또는 UDP
연결 상태

일반적으로 라우팅에 사용하지 않는 정보:

HTTP URL Path
HTTP Method
Cookie
HTTP Header
JSON Body

[L4 동작 예시]

클라이언트

| TCP 연결



L4 Load Balancer

|

└─ 10.0.0.11:443

└─ 10.0.0.12:443

└─ 10.0.0.13:443

클라이언트가 로드 밸런서의 203.0.113.10:443으로 연결하면 L4 로드 밸런서는 연결 정보와 알고리즘에 따라 Backend를 선택한다.

203.0.113.10:443



10.0.0.12:443

구현 방식에 따라 NAT 방식으로 패킷의 목적지 주소를 변경하거나, 별도의 TCP 연결을 만드는 프록시 방식으로 전달할 수도 있다.

따라서 “L4는 반드시 TCP 연결을 종료하지 않는다” 또는 “반드시 NAT만 사용한다”고 단정하면 안 된다. 핵심은 **Backend 선택이 전송 계층 정보를 기준으로 이루어진다는 것**이다.

[L4 장점]

- 패킷 내용을 깊게 분석하지 않아 처리 비용이 상대적으로 작음
- 대량 연결과 높은 처리량에 유리
- HTTP뿐 아니라 다양한 TCP·UDP 프로토콜 처리 가능
- 애플리케이션 프로토콜에 덜 의존함

사용할 수 있는 예시는 다음과 같다.

- HTTP/HTTPS
- TCP 데이터베이스 연결
- 게임 서버
- 메일 서버
- DNS UDP 트래픽
- 자체 TCP 프로토콜

[L4 단점]

- URL 경로 기반 분배가 어려움
- HTTP Header나 Cookie를 기준으로 분배하기 어려움
- 애플리케이션 수준의 세밀한 제어가 제한됨
- HTTP 요청 내용에 기반한 인증이나 필터링이 어려움

[10] [L7 Load Balancing]

L7 로드 밸런싱은 OSI 7계층인 애플리케이션 계층의 정보를 이해하고 요청을 분산한다. HTTP 환경에서는 다음 정보를 확인할 수 있다.

- Host
- URL Path
- HTTP Method
- Header
- Cookie
- Query Parameter
- Content-Type
- 일부 요청 Body 정보

```
GET /api/users HTTP/1.1
Host: api.example.com
Authorization: Bearer ...
Cookie: session=...
```

L7 로드 밸런서는 이러한 HTTP 정보를 이용하여 세밀한 라우팅을 할 수 있다.

[URL Path 기반 라우팅]

```
/api/users/* → User Server
/api/orders/* → Order Server
/images/* → Image Server
```

클라이언트 요청

|



L7 Load Balancer

|

```
├─ /users → User Service
├─ /orders → Order Service
└─ /images → Static Server
```

[Host 기반 라우팅]

```
api.example.com → API Server
admin.example.com → Admin Server
static.example.com → Static Server
```

하나의 로드 밸런서가 여러 도메인의 요청을 구분하여 서로 다른 Backend로 전달할 수 있다.

[Method 기반 라우팅]

필요하면 HTTP Method를 기준으로 다른 처리 정책을 적용할 수 있다.

GET 요청 → 조회 서버
POST·PUT 요청 → 쓰기 서버

다만 데이터 일관성과 서비스 설계를 고려해야 하므로 단순히 Method만 보고 서버를 분리하는 방식은 신중하게 사용해야 한다.

[Header·Cookie 기반 라우팅]

특정 헤더 또는 쿠키를 기준으로 Backend를 선택할 수도 있다.

X-App-Version: beta
→ 새로운 버전 서버

그 외 요청
→ 기존 버전 서버

이를 활용하여 다음 기능을 구현할 수 있다.

- A/B 테스트
- Canary 배포
- 특정 사용자만 새 버전 사용
- 모바일·웹 클라이언트 분리
- 세션 유지

[11] [L7 Reverse Proxy 구조]

L7 로드 밸런서는 일반적으로 Reverse Proxy로 동작한다.

클라이언트
| 연결 1
▼
L7 Load Balancer
| 연결 2
▼
Backend Server

클라이언트와 Backend가 직접 하나의 TCP 연결을 공유하는 것이 아니라 다음처럼 두 연결로 나뉠 수 있다.

클라이언트 ↔ 로드 밸런서
로드 밸런서 ↔ Backend

로드 밸런서는 클라이언트의 HTTP 요청을 해석한 후 적절한 Backend로 새로운 요청을 전달한다.



대표적인 L7 Reverse Proxy 소프트웨어로는 다음이 있다.

- Nginx
- HAProxy
- Envoy
- Traefik

HAProxy는 설정에 따라 L4와 L7 모드 모두 지원할 수 있다.

[12] [TLS Termination]

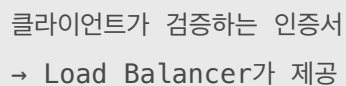
HTTPS 요청 내용을 확인하려면 암호문을 복호화해야 한다.

L7 로드 밸런서는 일반적으로 클라이언트와 TLS 연결을 맺고 HTTPS 요청을 복호화한다.

이를 TLS Termination 또는 TLS Offloading이라고 한다.



로드 밸런서가 서버 인증서와 개인키를 관리한다.



TLS를 종료한 뒤 Backend로 전달하는 방식은 두 가지가 있다.

[Backend에 HTTP로 전달]

클라이언트 — HTTPS —> Load Balancer
Load Balancer — HTTP —> Backend

내부 네트워크에서 암호화 비용을 줄일 수 있지만 내부 구간의 트래픽은 암호화되지 않는다.

[Backend에도 HTTPS로 전달]

클라이언트 — HTTPS —> Load Balancer
Load Balancer — HTTPS —> Backend

로드 밸런서가 클라이언트 TLS 연결을 종료한 후 Backend와 별도의 TLS 연결을 만든다.
이를 TLS Re-encryption 구조라고 부를 수 있다.

내부 구간까지 암호화할 수 있지만 인증서 관리와 암호화 처리 비용이 추가된다.

[13] [L4와 TLS]

L4 로드 밸런서는 HTTPS 내용을 복호화하지 않고 TCP 연결 정보를 바탕으로 Backend에 전달할 수 있다.

클라이언트 — 암호화된 TLS —> L4 Load Balancer
↓
Backend가 TLS 종료

이를 TLS Passthrough라고 한다.
이 경우 서버 인증서와 개인키는 Backend 서버가 관리할 수 있다.

장점:

- 로드 밸런서가 HTTP 내용을 볼 필요 없음
- TLS 암호화가 Backend까지 유지
- 로드 밸런서의 TLS 처리 부담 감소

단점:

- URL과 Header 기반 L7 라우팅이 제한됨
 - 중앙에서 HTTP 요청을 분석하거나 수정하기 어려움
 - Backend마다 인증서 관리가 필요할 수 있음
-

[14] [L4와 L7 비교]

구분	L4 Load Balancer	L7 Load Balancer
기준 계층	전송 계층	애플리케이션 계층
주요 판단 정보	IP, Port, TCP/UDP	Host, Path, Method, Header, Cookie
HTTP 내용 이해	일반적으로 하지 않음	이해하고 분석
처리 속도	상대적으로 빠름	상대적으로 처리 비용이 큼
라우팅 정밀도	낮음	높음
프로토콜 범위	다양한 TCP·UDP	HTTP 등 지원하는 애플리케이션 프로토콜
TLS Passthrough	적합	HTTP 분석이 어려움
TLS Termination	구현에 따라 가능	일반적으로 많이 사용
URL 기반 라우팅	불가능하거나 제한적	가능
Header 기반 라우팅	불가능하거나 제한적	가능
대표 용도	대량 TCP·UDP 연결 분산	웹·API 요청의 세밀한 분산

핵심 차이는 다음과 같다.

L4

- 어디에서 어디로 연결하는가?
- IP와 Port를 중심으로 판단

L7

- 어떤 HTTP 요청인가?
- Host, Path, Method, Header 등을 중심으로 판단

[15] [L4와 L7 요청 흐름 비교]

[L4]

클라이언트

| TCP 203.0.113.10:443



L4 Load Balancer

| IP·Port·Protocol 확인



10.0.0.12:443

L4는 다음 요청의 경로를 기준으로 라우팅하지 않는다.

```
GET /users
GET /orders
```

둘 다 동일한 443 포트의 연결이라면 전송 계층에서는 같은 종류의 트래픽으로 볼 수 있다.

[L7]

```
클라이언트
  | HTTPS Request
  ▼
L7 Load Balancer
  | HTTP 요청 분석
  ├── /users → User Server
  └── /orders → Order Server
```

[16] [Session Persistence]

로드 밸런서는 같은 사용자의 요청을 계속 같은 Backend로 보내야 하는 경우가 있다.

이를 Session Persistence 또는 Sticky Session이라고 한다.

```
사용자 A 요청 1 → 서버 A
사용자 A 요청 2 → 서버 A
사용자 A 요청 3 → 서버 A
```

Sticky Session을 구현하는 기준에는 다음과 같은 방법이 있다.

- Source IP Hash
- Cookie
- Session ID
- 특정 HTTP Header

[왜 필요한가?]

세션 정보를 각 Backend 서버의 메모리에만 저장했다고 하자.

```
로그인 요청 → 서버 A
세션 정보 → 서버 A 메모리에 저장

다음 요청 → 서버 B
서버 B에는 세션 정보 없음
→ 로그인이 풀린 것처럼 보임
```

Sticky Session을 사용하면 같은 사용자를 서버 A로 계속 보낼 수 있다.

[단점]

- 특정 서버에 사용자가 집중될 수 있음
- 서버 A 장애 시 해당 세션이 사라질 수 있음
- 서버 확장 및 축소가 복잡해질 수 있음

따라서 일반적으로는 세션을 외부 저장소에 공유하는 구조도 많이 사용한다.

```
서버 A └─┐
서버 B ──┴──> Redis Session Store
서버 C └─┐
```

또는 JWT처럼 서버가 세션 상태를 직접 저장하지 않는 방식을 사용할 수 있다.

[17] [클라이언트 IP 전달]

L7 Reverse Proxy 구조에서는 Backend가 직접 본 TCP 상대방이 클라이언트가 아니라 로드 밸런서일 수 있다.

```
클라이언트 — TCP 연결 1 —> 로드 밸런서
로드 밸런서 — TCP 연결 2 —> 백엔드
```

백엔드와 직접 TCP 연결을 맺은 상대는 로드 밸런서이므로 백엔드가 소켓에서 확인하는 출발지 IP는 다음과 같다.

```
Remote Address = 로드 밸런서 IP
```

실제 클라이언트 IP는 HTTP 헤더를 통해 별도로 전달해야 한다.

X-Forwarded 헤더

```
GET /login HTTP/1.1
Host: internal-backend:8080
X-Forwarded-For: 203.0.113.50
X-Forwarded-Proto: https
X-Forwarded-Host: example.com
```

헤더	전달하는 정보
X-Forwarded-For	실제 클라이언트 IP
X-Forwarded-Proto	클라이언트가 프록시까지 사용한 프로토콜
X-Forwarded-Host	클라이언트가 요청한 원래 도메인
X-Forwarded-Port	클라이언트가 요청한 원래 포트

X-Forwarded-Proto: https는 백엔드까지도 HTTPS라는 뜻이 아니다. 다음과 같은 구성도 가능하다.

클라이언트 — HTTPS —> 로드 밸런서 — HTTP —> 백엔드

로드 밸런서에서 TLS를 종료했기 때문에 백엔드는 HTTP 요청을 받지만, 사용자가 원래 사용한 프로토콜은 https 였다는 것을 알려주는 것이다.

[프록시가 여러 개라면]

여러 프록시를 거치면 X-Forwarded-For에 IP가 순서대로 추가될 수 있다.

X-Forwarded-For: 203.0.113.50, 10.0.1.10, 10.0.2.20

일반적인 의미는 다음과 같다.

203.0.113.50 → 실제 클라이언트
10.0.1.10 → 첫 번째 프록시
10.0.2.20 → 두 번째 프록시

하지만 단순히 가장 왼쪽 값을 무조건 실제 IP로 믿으면 안된다.

[보안상 주의점]

외부 사용자가 직접 다음 헤더를 넣을 수 있기 때문이다.

X-Forwarded-For: 127.0.0.1

백엔드가 이것을 그대로 믿으면 공격자가 내부 사용자인 것처럼 위장할 수 있다.

따라서 다음 구조로 운영해야 한다.

1. 백엔드는 신뢰할 수 있는 프록시를 통해서만 접근하게 한다.
2. 가장 앞단의 프록시는 외부에서 받은 위조 헤더를 삭제하거나 올바르게 재작성한다.
3. 백엔드는 설정된 신뢰 프록시가 전달한 헤더만 해석한다.
4. IP 기반 인증이나 권한 판단은 특히 신중하게 사용한다.

즉, 헤더 자체를 신뢰하는 것은 아니라 그 헤더를 전달할 프록시를 신뢰하는 것이다.

[L4의 Proxy Protocol]

L4 로드 밸런서는 HTTP 내용을 해석하지 않으므로 일반적으로 HTTP 헤더를 추가할 수 없다.

대신 TCP 데이터 앞부분에 원래 주소 정보를 붙이는 PROXY Protocol을 사용할 수 있다.

PROXY TCP4 203.0.113.50 10.0.0.20 54321 443

- 원래 클라이언트 IP와 포트
- 목적지 서버 IP와 포트
- IPv4 또는 IPv6 같은 연결 정보

단, 백엔드도 PROXY Protocol을 지원하고 사용하도록 설정되어 있어야 한다.

지원하지 않으면 이 정보를 애플리케이션 데이터로 오인한다.

[18] [로드 밸런서의 장애]

로드 밸런서 한 대만 사용하면 로드 밸런서가 새로운 단일 장애 지점이 될 수 있다.

Load Balancer 장애

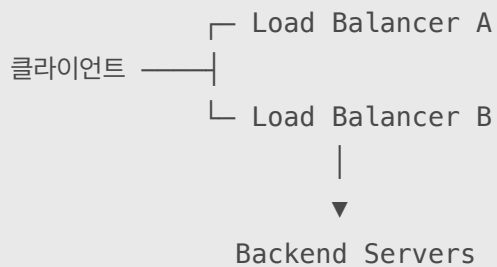


모든 Backend가 정상이어도

클라이언트가 접근하지 못함

이를 SPOF(Single Point of Failure)라고 한다.

따라서 실제 서비스에서는 로드 밸런서 자체도 이중화할 수 있다.



클라우드 관리형 로드 밸런서는 서비스 제공자가 여러 장비와 장애 조치 구조를 관리한다.

[19] [Nginx Round Robin 예시]

Nginx에서는 기본적인 HTTP Upstream을 다음처럼 구성할 수 있다.

```
upstream backend_servers {
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
    server 10.0.0.13:8080;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend_servers;
    }
}
```

별도 알고리즘을 지정하지 않으면 일반적으로 Round Robin 방식으로 분산한다.

요청 1 → 10.0.0.11

요청 2 → 10.0.0.12

요청 3 → 10.0.0.13

[20] [Nginx Least Conneciton 예시]

```
upstream backend_servers {  
    least_conn;  
  
    server 10.0.0.11:8080;  
    server 10.0.0.12:8080;  
    server 10.0.0.13:8080;  
}
```

새 요청은 활성 연결 수가 상대적으로 적은 서버로 전달된다.

가중치를 함께 지정할 수도 있다.

```
upstream backend_servers {  
    least_conn;  
  
    server 10.0.0.11:8080 weight=3;  
    server 10.0.0.12:8080 weight=2;  
    server 10.0.0.13:8080 weight=1;  
}
```

[21] [Nginx L7 경로 라우팅 예시]

```
server {  
    listen 80;  
  
    location /api/users/ {  
        proxy_pass http://user_servers;  
    }  
  
    location /api/orders/ {  
        proxy_pass http://order_servers;  
    }  
  
    location /static/ {  
        proxy_pass http://static_servers;  
    }  
}
```

```
/api/users  → user_servers  
/api/orders → order_servers  
/static     → static_servers
```

Nginx가 HTTP 경로를 확인하므로 L7 라우팅에 해당한다.